

Laboratório 01

26/05/2026 (valendo 10 pontos)

Ambientação

1. Insomnia
2. Python
3. Django
4. MySQL
5. Postgree



insomnia



 Welcome to Insomnia 12!

This new version of Insomnia is the biggest one ever! Notable new features that we have shipped are:



MCP Client



Auto-generated mock servers



Smart Commits



Git Sync for all plans

Continue

 Home

 Organizations

 Plan & billing

Welcome to Insomnia, romulo!

Recent organizations [View all](#)

Personal Workspace



 Open

Launch Insomnia desktop



[Manually enter token](#)

Launch

Your Invi

No pending invitations



In

 Home Organizations Plan & billing

Welcome to Insomnia, romulo!

Recent organizations [View all](#)

Personal Workspace

 Open

Your Invitations (0)

No pending invitations

Helpful Resources

-  [Download desktop app](#)
-  [Insomnia documentation](#)
-  [Inso CLI](#)

Essentials Plan

[Upgrade](#)Users 

1/Unlimited

Mock Requests 

0/1,000

GET

get_paises_ibge

GET get_paises_ibge ×

+

GET https://servicodados.ibge.gov.br/api/v1/localidades/paises

Send

200 OK

227 ms

44.4 KB

Params

Body

Auth

Headers

3

Scripts

Docs

Preview

Headers

8

Cookies

URL PREVIEW

https://servicodados.ibge.gov.br/api/v1/localidades/paises



QUERY PARAMETERS

Import from URL



Bulk Edit

+ Add



Delete all



Description

name

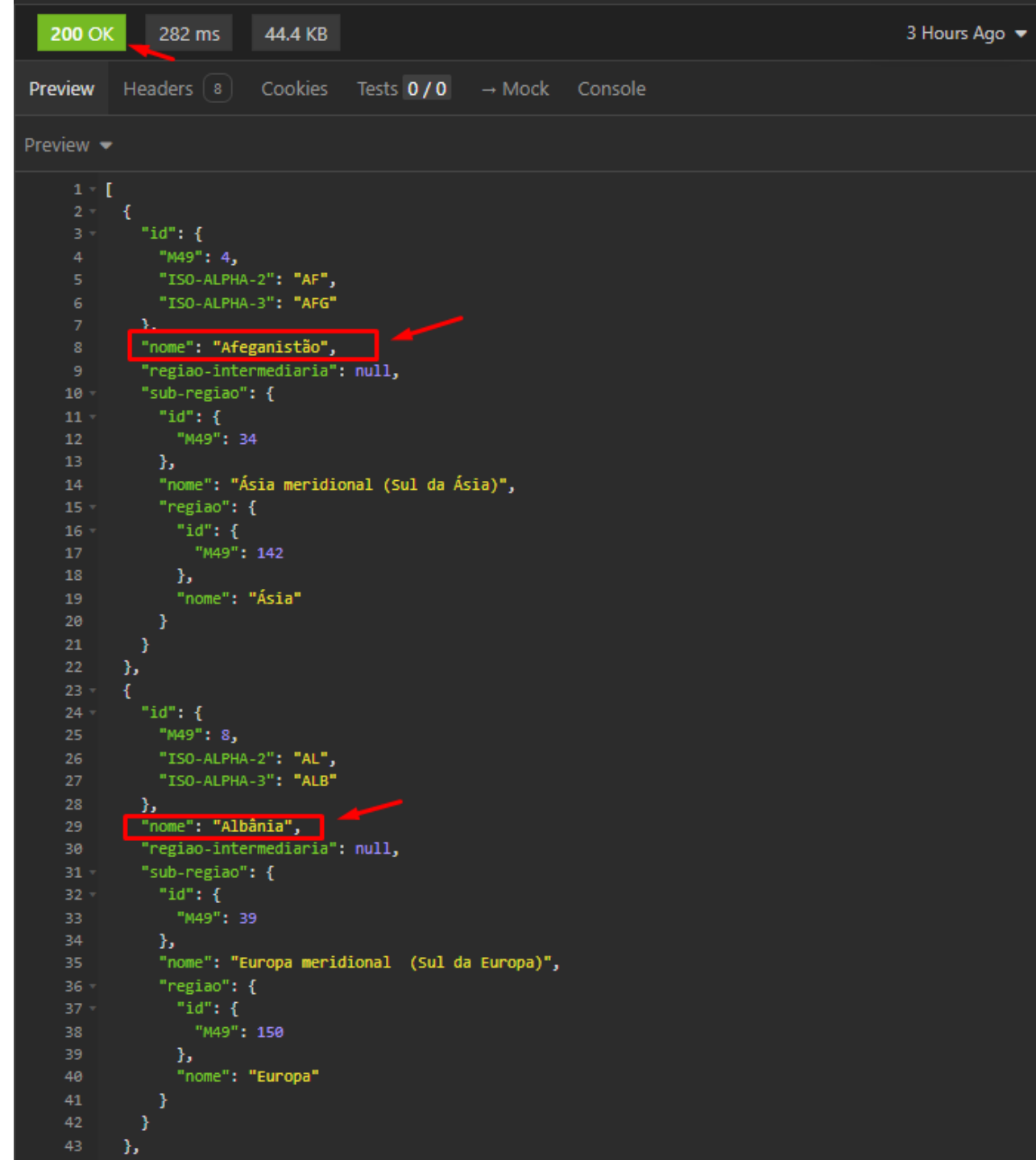
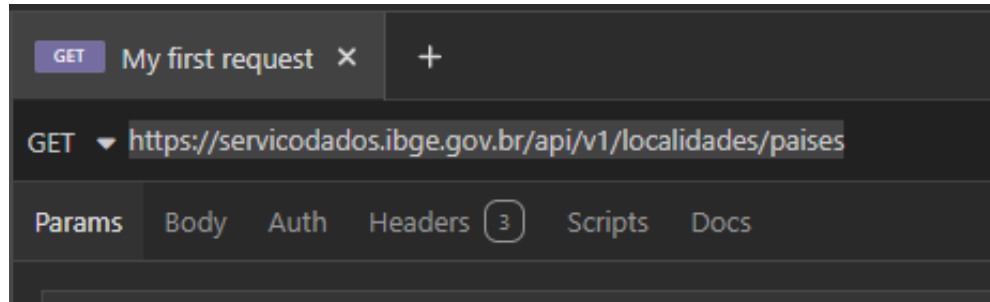
value



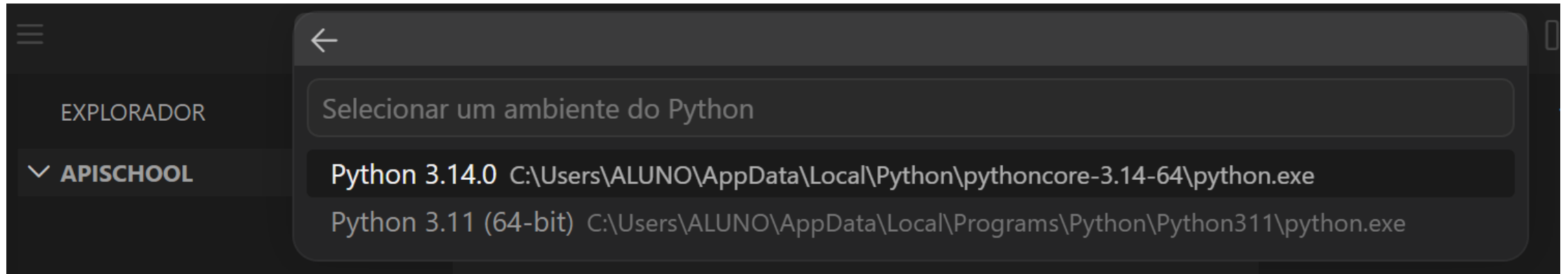
Preview

```
481     }
482   },
483   {
484     "id": {
485       "M49": 84,
486       "ISO-ALPHA-2": "BZ",
487       "ISO-ALPHA-3": "BLZ"
488     },
489     "nome": "Belize",
490     "regiao-intermediaria"
491     "id": {
492       "M49": 13
493     },
494     "nome": "América cen
```

Instalação e Teste do Insomnia



Qual a versão python esta instalada na máquina...





Create your account

Username:

Email:

Password:

Password (again):

☐ I agree to the [Terms and Conditions](#) and the [Privacy and Cookies Policy](#),
and confirm that I am at least 13 years old.

Register

We promise not to spam or pass your details on to anyone else.



[Dashboard](#) [Consoles](#) [Files](#) [Web](#) [Tasks](#) [Databases](#)

Dashboard

Welcome, [raquelleite](#)

CPU Usage: 0% used – 0.00s of 100s. Resets in 23 hours, 58 minutes [More Info](#)

[Upgrade Account](#)

File storage: 0% full – 60.0 KB of your 512.0 MB quota [More Info](#)

Recent
Consoles

+ 5 -

You have no recent consoles.

New console:

\$ Bash

>>> Python ▾

[More...](#)

Recent
Files

+ 5 -

You have no recently edited files.

[+ Open another file](#)

[Browse files](#)

Recent
Notebooks

+ 5 -

Your account does not support Jupyter Notebooks. [Upgrade your account](#) to get access!

All
Web apps

You don't have any web apps.

[Open Web tab](#)



MySQL

[Postgres](#)

Upgrade required

MySQL is not available on free accounts. If you want to enable it, you'll have to [upgrade](#) to any paid plan.

 No cloud (sem AWS, sem Azure) gratuito

 PythonAnywhere

IDE + hospedagem de apps
Flask/Django direto no
navegador. Ideal para aulas de
Python sem instalar nada.

1 web app · 512MB · 100 CPU-seg/
dia

⚠ Desde jan/2026: sem suporte
direto no plano free

 Vercel

Especialista em React/Next.js.
Deploy via Git com CDN global.
Perfeito para projetos de front-
end.

Sites estáticos ilimitados ·
serverless

 Netlify

Sites estáticos (JAMstack),
Hugo, Gatsby, Astro. Fácil
integração com GitHub.

100GB banda · deploy
automático

 Render

Full-stack: web services, banco
PostgreSQL, sites estáticos. Sem
cartão de crédito.

750h/mês · PostgreSQL 90 dias

⚠ Serviço dorme após 15min
inativo

 Railway

Backend full-stack com banco
integrado. Excelente para
protótipos rápidos e side
projects.

\$5 crédito/mês (~500h) · trial-
based

 GitHub Pages

Hospedagem estática direto do
repositório. Zero configuração.
Ideal para portfólios e
documentação.

Gratuito · Integrado ao GitHub

 Firebase Hosting

Google. Hosting estático + Cloud
Functions (Node.js). Integra com
Firestore e Auth.

10GB armazenamento · 360MB/
dia

 GoogieHost

Hospedagem PHP + MySQL com
cPanel. Voltado para estudantes
e iniciantes sem habilidades de
código.

SSD ilimitado · domínio grátis



AWS — planos para estudantes

2 programas disponíveis



AWS Educate

Laboratórios sandbox pré-configurados com EC2, S3, RDS, VPC, Lambda e DynamoDB. Sem cartão de crédito. A partir de 13 anos, apenas com e-mail.

Labs guiados · sem cobrança acidental

Foco em aprendizagem supervisionada, não deploy livre.



AWS Free Tier

EC2 t2.micro (750h/mês), S3 (5GB), RDS (750h), Lambda (1M req/mês), Bedrock. Novo: \$200 em créditos para novos cadastros.

12 meses + \$200 créditos iniciais



Requer cartão de crédito no cadastro



AWS Academy

Curriculo oficial para instituições de ensino superior. O professor cadastra a turma e os alunos recebem créditos para laboratórios.

Acesso institucional · via professor

Dica para turmas: O AWS Educate é o mais indicado para alunos — sem cartão, sem risco de cobrança e com labs estruturados para EC2, S3, RDS e VPC.



Azure — planos para estudantes

2 planos disponíveis



Azure for Students

\$100 em créditos por 12 meses (renovável). Acesso a 25+ serviços: VMs Linux/Windows, App Service, SQL Database, Blob Storage, Cognitive Services.

\$100 · 12 meses · sem cartão

Requer e-mail institucional válido. Renova anualmente.



Azure for Students Starter

Versão sem créditos e sem prazo. Acesso gratuito permanente a App Service, Azure Functions, Cognitive Services e Cosmos DB (limitado).

Gratuito · sem prazo · ensino médio+

Ideal para alunos sem e-mail universitário.

Vantagem Azure vs AWS para alunos: O Azure for Students não exige cartão de crédito em nenhum dos dois planos — diferencial importante para turmas iniciantes.

On-Promisse

- Servidor criado na própria empresa.
- Ver com Suporte - WS
- Acessar carreta

E aí?

- **PythonAnywhere** — suporte nativo a Django e MySQL, tudo no navegador. É o mais indicado para ensino justamente por essa combinação pronta. ⚠️ Contas novas (após jan/2026) não têm mais MySQL no plano gratuito — só contas antigas mantêm o banco.
- **Render** — suporta Django via Docker ou buildpack Python. O banco gratuito é PostgreSQL (não MySQL nativamente), mas aceita MySQL externo via variável de ambiente.
- **Railway** — suporta Django e tem plugin de MySQL disponível. O plano é crédito-based (\$5/mês), não puramente gratuito.
- **AWS Educate / Free Tier** — EC2 + RDS (MySQL disponível). No Free Tier, o RDS MySQL tem 750h/mês gratuitas por 12 meses. No Educate, o acesso ao RDS depende do template do professor.
- **Azure for Students** — App Service (suporta Django via Python) + Azure Database for MySQL. Os \$100 de crédito cobrem os dois serviços tranquilamente por meses.

[+ Add a new web app](#)Create new web app 

Select a Python Web framework

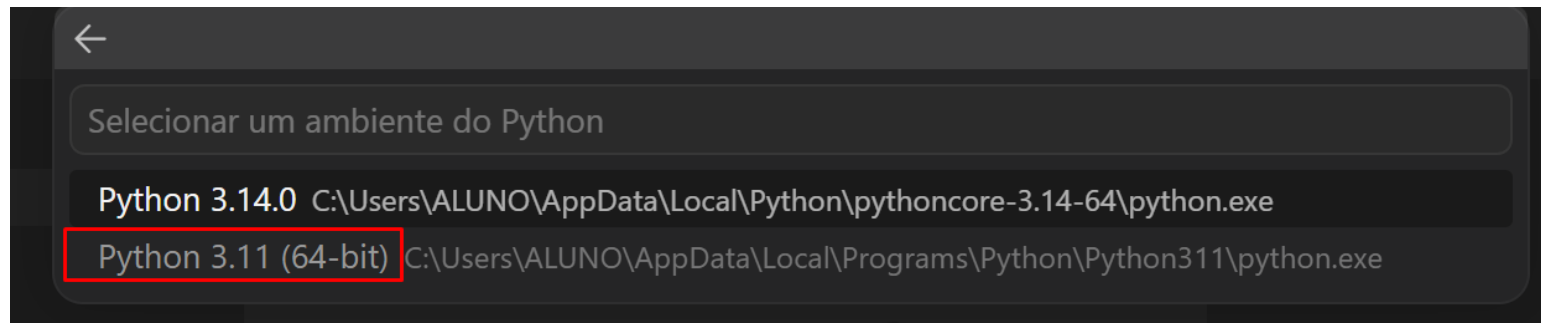
...or select "Manual configuration" if you want detailed control.

- » Django
- » web2py
- » Flask
- » Bottle
- » **Manual configuration** (including virtualenvs)

What other frameworks should we have here? Send us some feedback using the link at the top of the page!

[Cancel](#)[« Back](#)[Next »](#)

+ Add a new web app



Create new web app

Select a Python version

- » Python 3.9 (Django 4.2.16)
- » Python 3.10 (Django 5.0.9)
- » Python 3.11 (Django 5.0.9)
- » Python 3.12 (Django 5.0.9)
- » Python 3.13 (Django 5.1.3)

Note: If you'd like to use a different version of Django to the default version, you can use a virtualenv for your web app. There are [instructions here](#).

Cancel

« Back

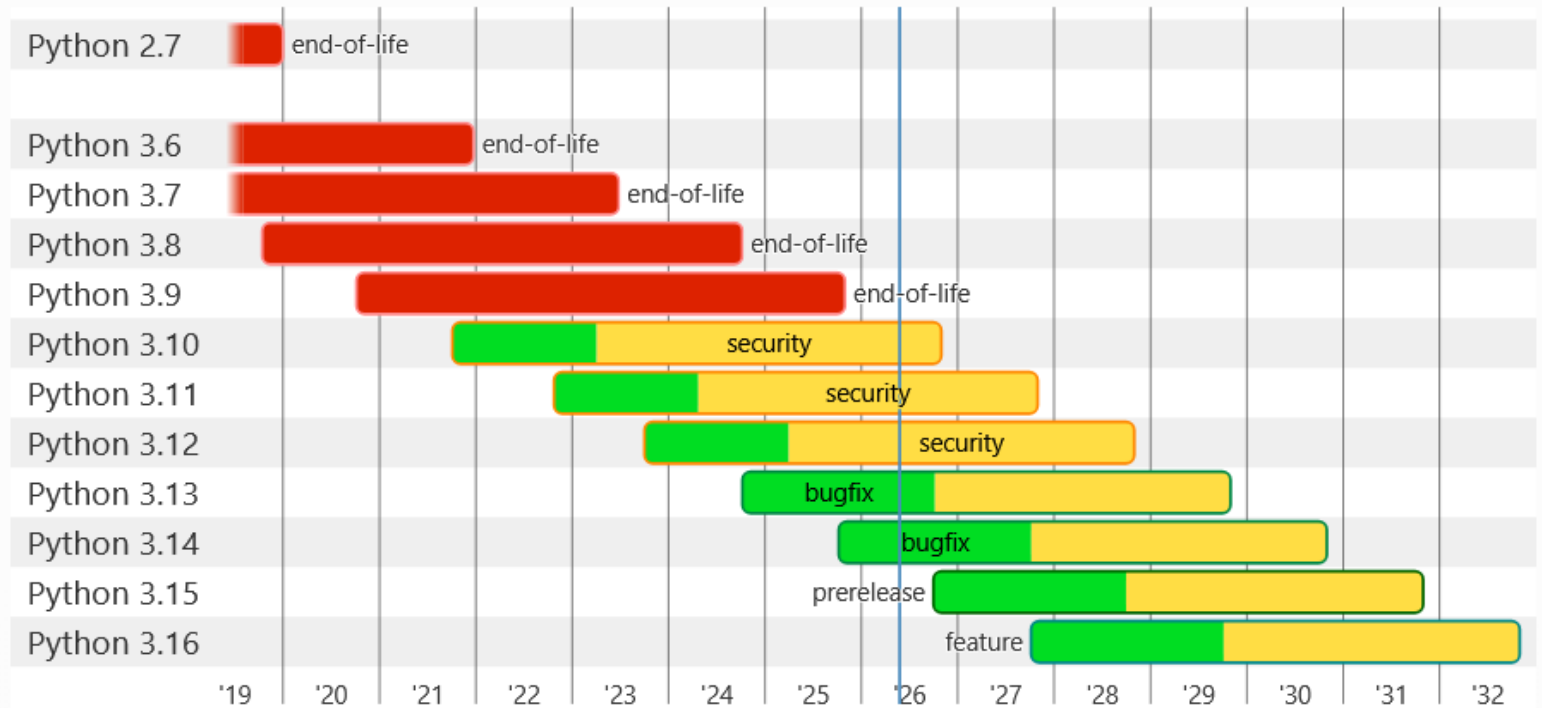
Next »

Vamos de...

- Python 3.13
- Django 5.1.3

Active Python releases

[For more information visit the Python Developer's Guide.](#)



Python version	Maintenance status		First released	End of support	Release schedule
3.16	pre-release	 Download	2027-10-06 (planned)	2032-10	PEP 826
3.14	bugfix	 Download	2025-10-07	2030-10	PEP 745
3.13	bugfix	 Download	2024-10-07	2029-10	PEP 719
3.12	security	 Download	2023-10-02	2028-10	PEP 693
3.11	security	 Download	2022-10-24	2027-10	PEP 664
3.10	security	 Download	2021-10-04	2026-10	PEP 619
3.9	end-of-life, last release was 3.9.25	 Download	2020-10-05	2025-10-31	PEP 596

<https://www.python.org/downloads/release/python-3130/>



Install Python 3.13.0 (64-bit)

Select **Install Now** to install Python with default settings, or choose **Customize** to enable or disable features.

→ **Install Now**

C:\Users\ALUNO\AppData\Local\Programs\Python\Python313

Includes IDLE, pip and documentation
Creates shortcuts and file associations

→ **Customize installation**

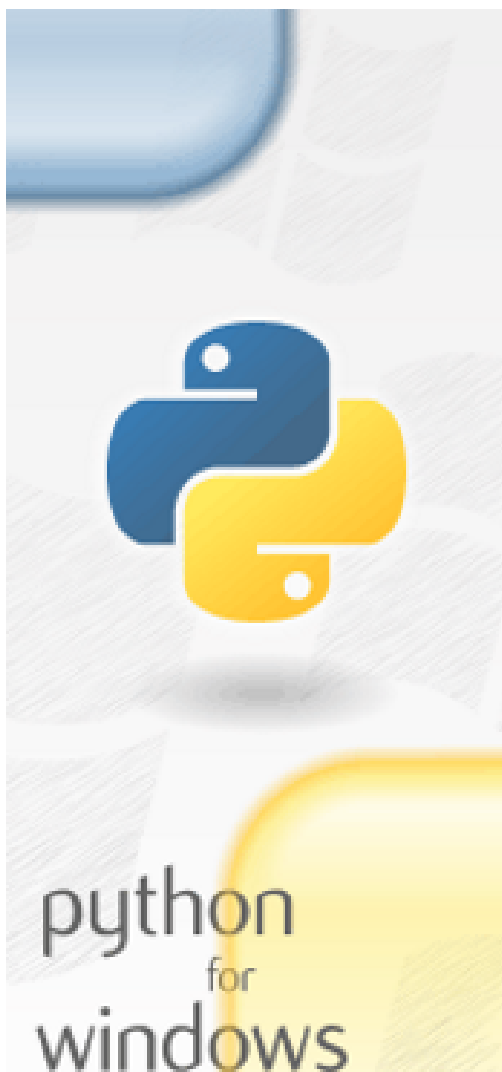
Choose location and features

☐ Use admin privileges when installing py.exe

☒ **Add python.exe to PATH**



Cancel



Setup was successful

New to Python? Start with the [online tutorial](#) and [documentation](#). At your terminal, type "py" to launch Python, or search for Python in your Start menu.

See [what's new](#) in this release, or find more info about [using Python on Windows](#).



Disable path length limit

Changes your machine configuration to allow programs, including Python, to bypass the 260 character "MAX_PATH" limitation.

Close



Selecionar um ambiente do Python

Python 3.14.0 C:\Users\ALUNO\AppData\Local\Python\pythoncore-3.14-64\python.exe

Python 3.13 (64-bit) C:\Users\ALUNO\AppData\Local\Programs\Python\Python313\python.exe

Python 3.11 (64-bit) C:\Users\ALUNO\AppData\Local\Programs\Python\Python311\python.exe

✓ APISCHOOL

>  .venv

EXPLORADOR

...

 helloapi.py X

▼ APISCHOOL

>  .venv

 helloapi.py

 helloapi.py

1 `print("Hello World API!")`

```
(.venv) PS C:\Users\ALUNO\Desktop\apischool> python helloapi.py  
Hello World API!
```

```
pip install django==5.1.3
```

pip list

```
(.venv) C:\Users\ALUNO\Desktop\apischool>pip list
```

Package	Version
-----	-----
asgiref	3.11.1
Django	5.1.3
pip	26.1.1
sqlparse	0.5.5
tzdata	2026.2

```
pip freeze > requirements.txt
```

✓ APISCHOOL

>  .venv

 helloapi.py

 requirements.txt

EXPLORADOR



.gitignore



✓ APISCHOOL



.venv



.gitignore



helloapi.py



requirements.txt



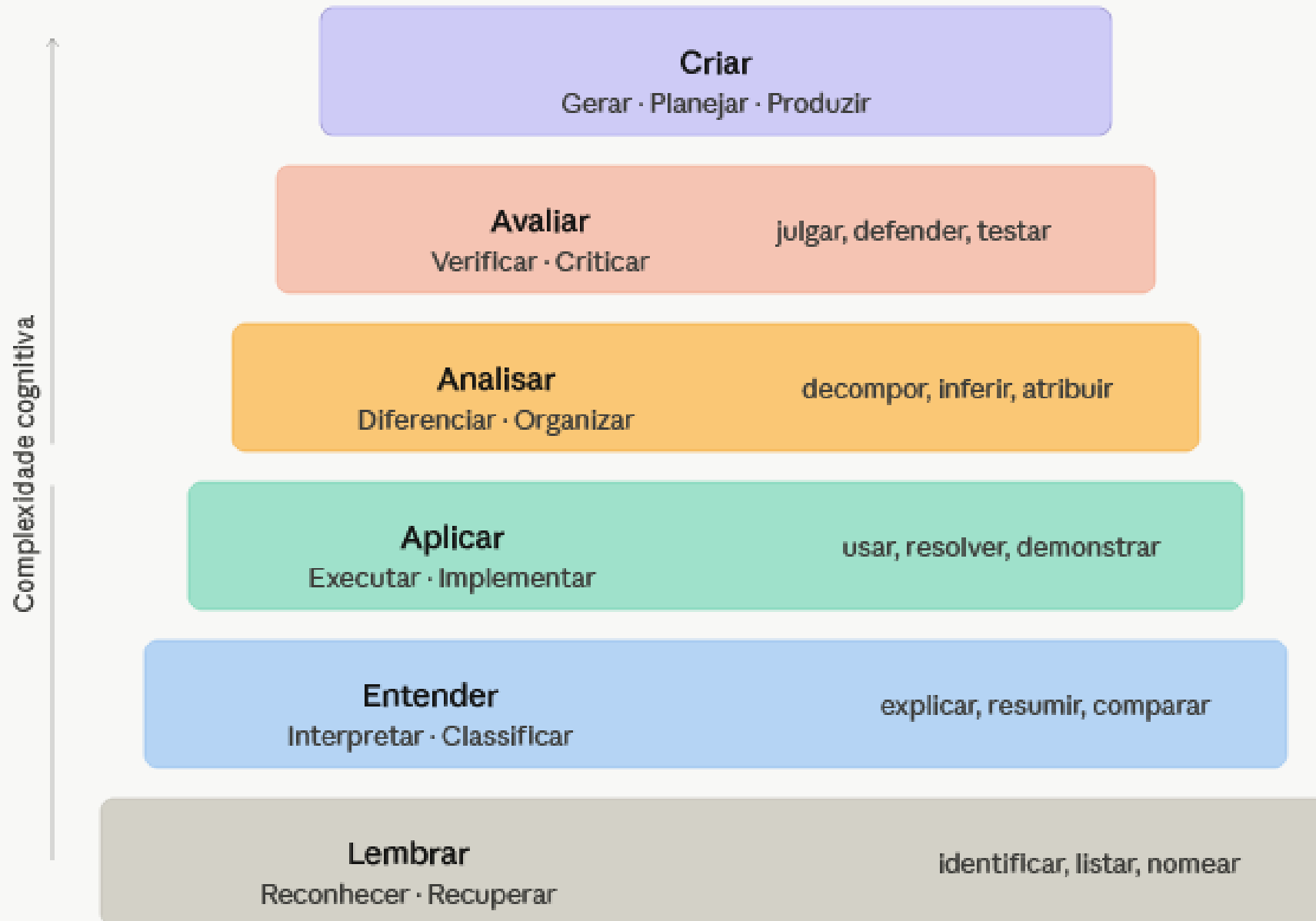
.gitignore

1

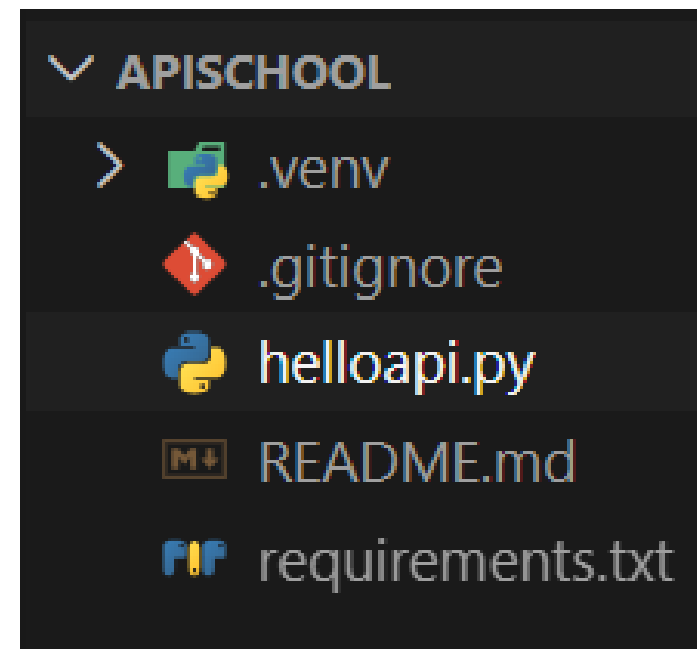
venv

2

.venv



Entender	Explicar a importância do ambiente virtual (venv) no isolamento de dependências
Aplicar	Criar e ativar um ambiente virtual com <code>venv</code>
Aplicar	Configurar um arquivo <code>.gitignore</code> para ignorar arquivos desnecessários no repositório
Aplicar	Desenvolver um arquivo <code>helloapi.py</code> com uma rota básica funcional
Aplicar	Gerar o arquivo <code>requirements.txt</code> com as dependências do projeto
Entender	Descrever a função de um <code>README.md</code> e sua relevância para repositórios profissionais
Aplicar	Escrever um <code>README.md</code> com estrutura adequada para o projeto



AMBIENTAÇÃO

3 ...

SQLite

MYSQL

PostgreSQL

+ Adicionar um cartão



BACKLOG

0 ...

+ Adicionar um cartão



DESENVOLVENDO

0 ...

+ Adicionar um cartão



FINALIZADO

4 ...

Insomnia

Python

Django

PythonAnywhere (conta)

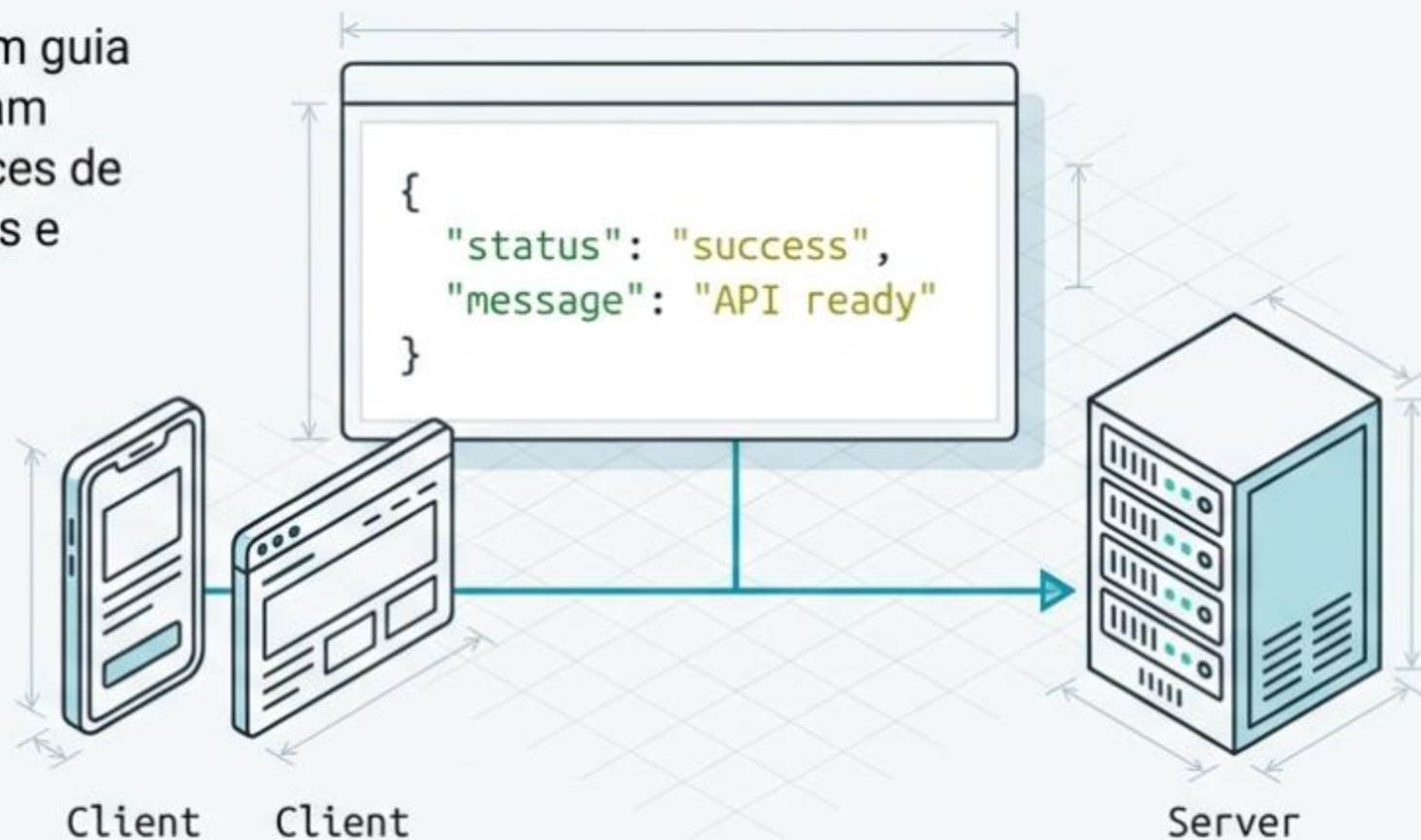
+ Adicionar um cartão

[aceite o convite aqui](#)

Laboratório 02 (01 de Junho)

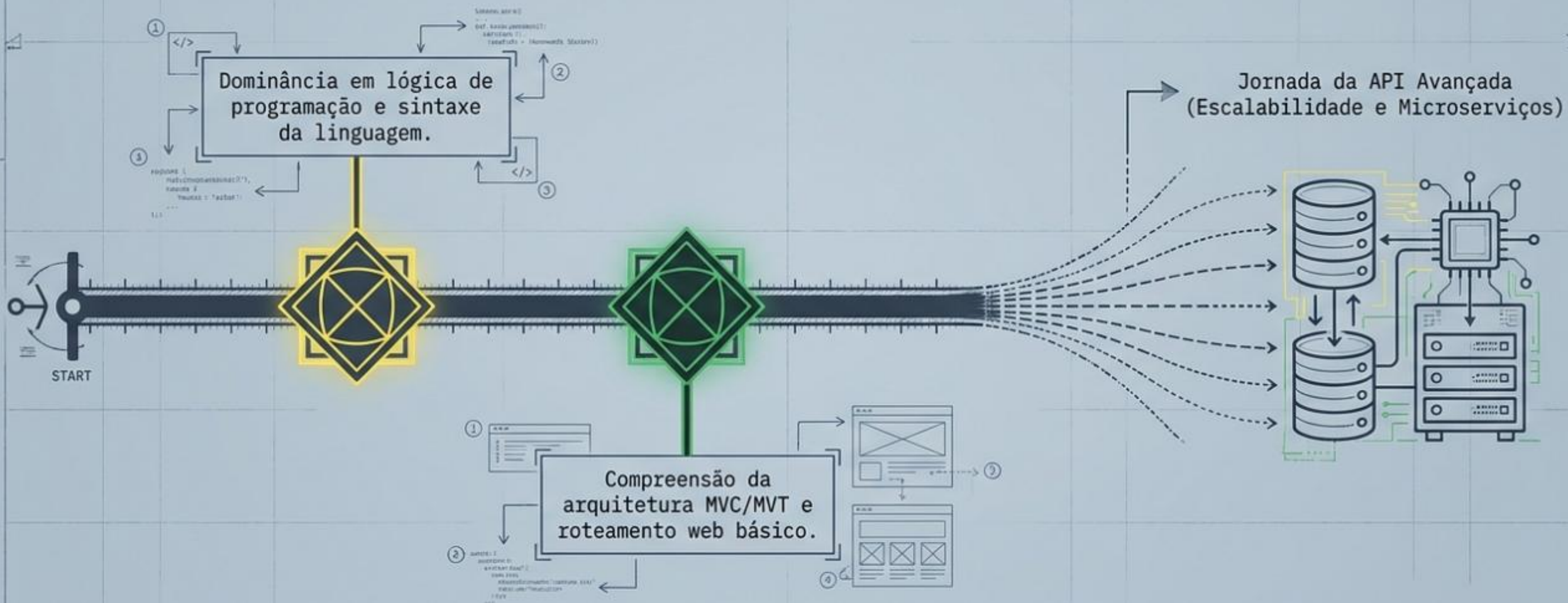
Construindo APIs Modernas com Django Rest Framework

Domine a arquitetura backend moderna. Um guia definitivo para desenvolvedores que desejam transformar lógica de negócios em interfaces de programação de aplicações (APIs) robustas e escaláveis.



0 Ponto de Partida Tecnológico

A construção de APIs avançadas exige alicerces sólidos. Este ecossistema foi desenhado para quem já domina Python e possui familiaridade com o ecossistema base do framework Django.
A partir daqui, mudamos o paradigma.



Do Monolito à Arquitetura Desacoplada

O Django padrão resolve a exibição na web. O DRF transforma sua aplicação em um motor de dados universal, capaz de alimentar qualquer interface gráfica moderna.

Django Clássico

Saída (Output)	Renderiza páginas HTML
Foco	Aplicações web monolíticas (Front e Back integrados)
Comunicação	Navegadores web tradicionais

Django Rest Framework (DRF)

Saída (Output)	Retorna dados puros (JSON ou XML)
Foco	APIs desacopladas para múltiplos clientes
Comunicação	Aplicativos mobile, SPAs (React/Vue) e integrações de sistemas

Encontre, instale e publique pacotes Python
com o Python Package Index

Digite



[PyPI · O Python Package Index](#)

django

The web framework for
perfectionists with deadlines.

[The web framework for perfectionists with deadlines | Django](#)

django

REST

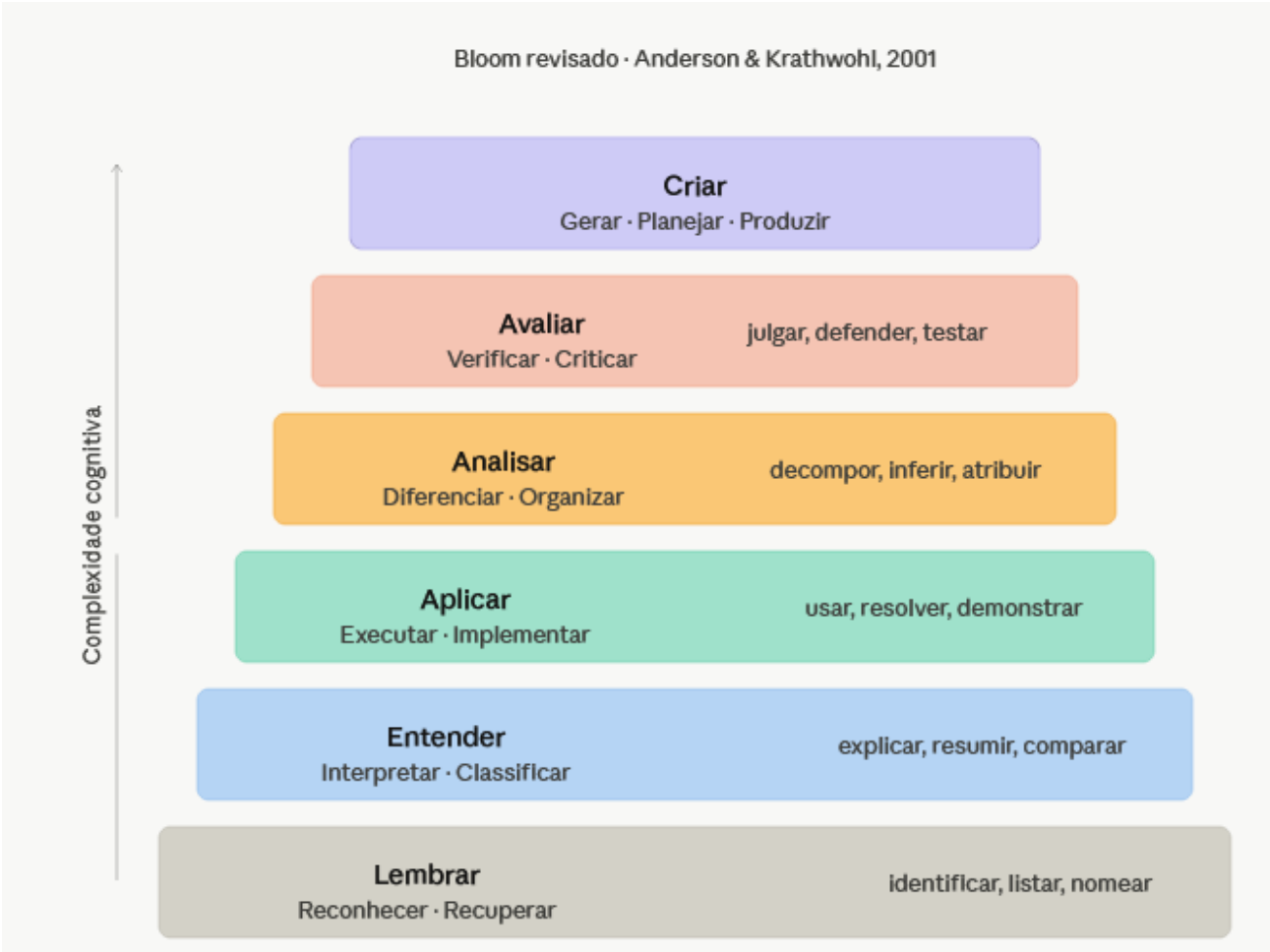
framework

[Django REST framework](#)

```
... pip freeze > requirements.txt  
C:\>django-admin startproject setup .
```



```
✓ APIBOX
> .venv
> setup
dj manage.py
requirements.txt
```



```
python manage.py startapp apibox_app
```

✓  apibox_app

>  migrations

 __init__.py

 admin.py

 apps.py

 models.py

 tests.py

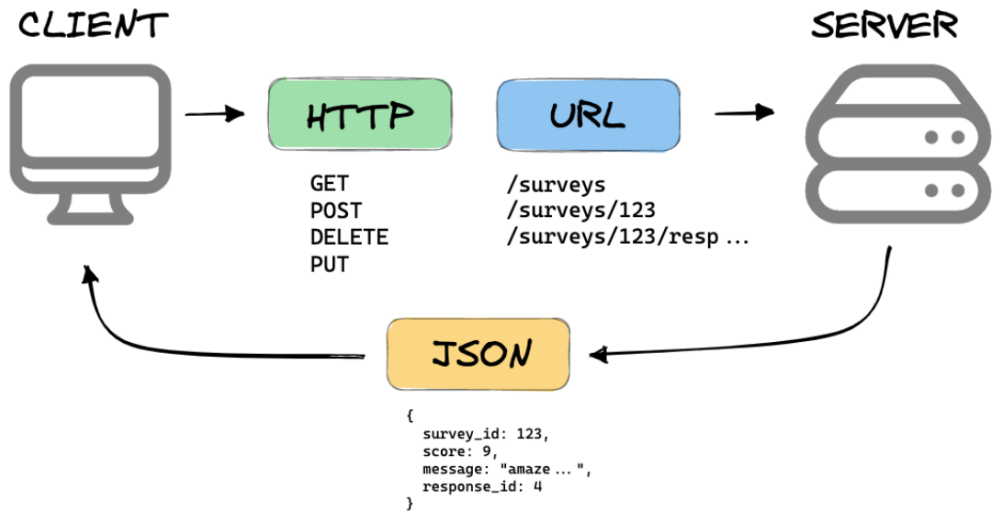
 views.py

```
33 INSTALLED_APPS = [  
34     'django.contrib.admin',  
35     'django.contrib.auth',  
36     'django.contrib.contenttypes',  
37     'django.contrib.sessions',  
38     'django.contrib.messages',  
39     'django.contrib.staticfiles',  
40     'apibox_app',  
41 ]  
42
```

```
LANGUAGE_CODE = 'pt-br'
```

```
TIME_ZONE = 'America/Sao_Paulo'
```

WHAT IS A REST API?



mannhowie.com

```
8 from django.http import JsonResponse
9
10 def box(request):
11     if request.method == 'GET':
12         box_feira={
13             'id': 1,
14             'nome': 'Loja Infantil Kids',
15             'numero': 101,
16         }
17     return JsonResponse(box_feira)
```

(module) http

```
from django.http import JsonResponse
```

"JsonResponse" is not

(class) JsonResponse

An HTTP response class
serialized to JSON.

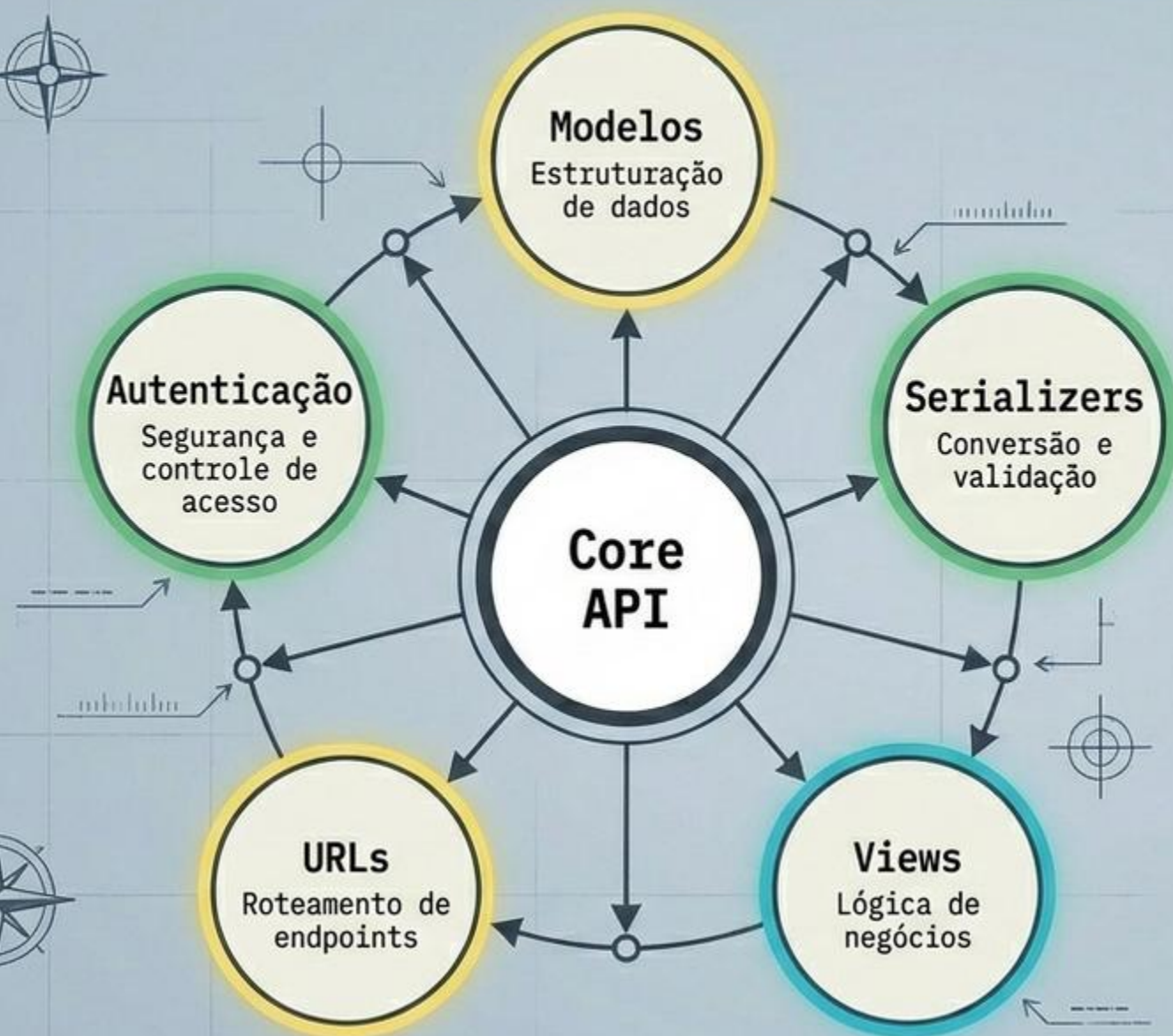
Parameters

JsonResponse

A Anatomia de uma API Restful

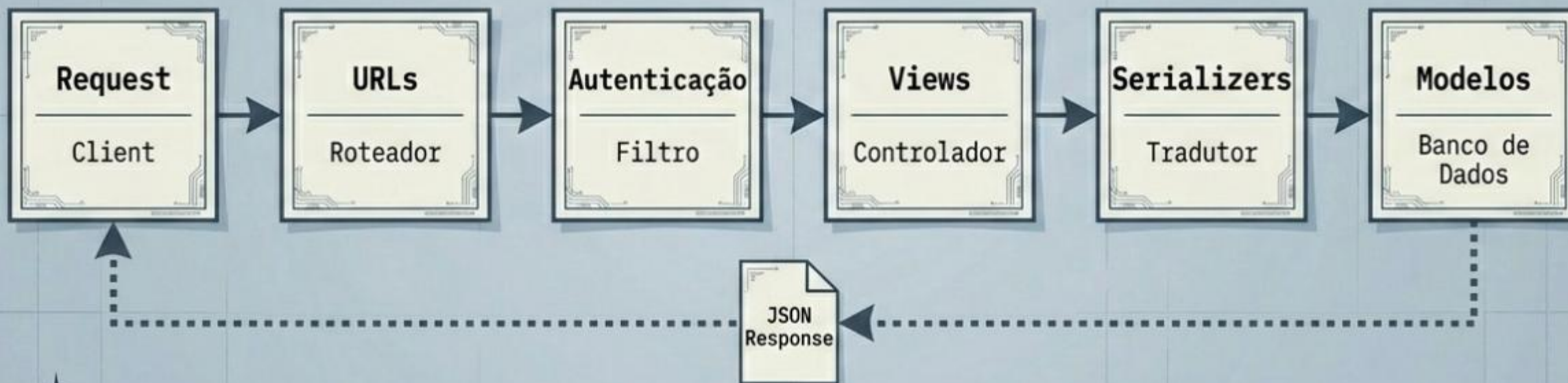
PROJETO:	
APROVADO:	
DESENVOLVIDOR:	
REVISÃO:	

Desenvolveremos uma arquitetura completa do zero. Cada módulo deste programa é uma peça fundamental no ciclo de vida de uma requisição web moderna.



O Ciclo de Vida da Requisição

Antes de escrevermos a primeira linha de código, este é o mapa mental que governará toda a nossa jornada de desenvolvimento.

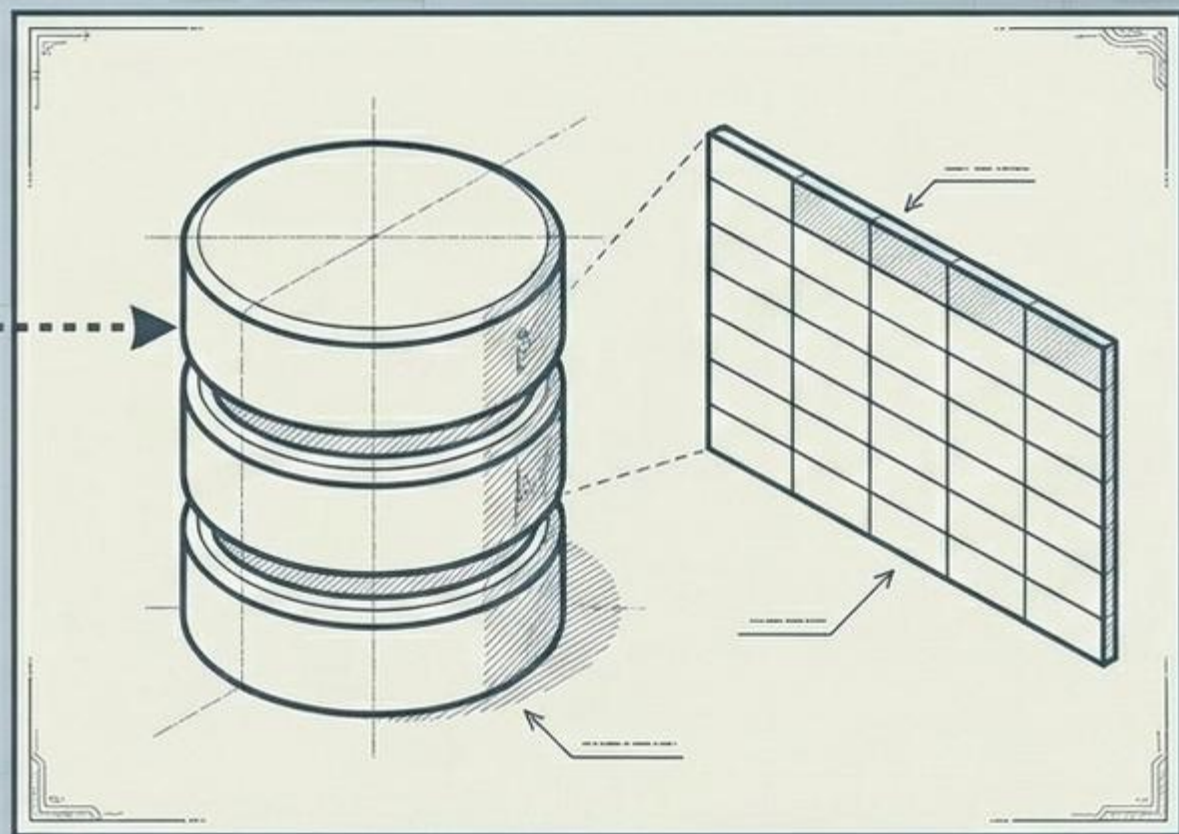


Modelos: A Fundação dos Dados



Tudo começa com a definição clara das entidades. Aprenderemos a mapear a realidade para dentro do banco de dados usando o ORM do Django, criando estruturas persistentes, relacionamentos e restrições de integridade.

```
class Estudante(models.Model):  
    nome = models.CharField(max_length=100)  
    matricula = models.IntegerField(unique=True)
```





JSON

JSON stands for **JavaScript Object Notation**.

JSON is a **plain text format** for storing and transporting data.

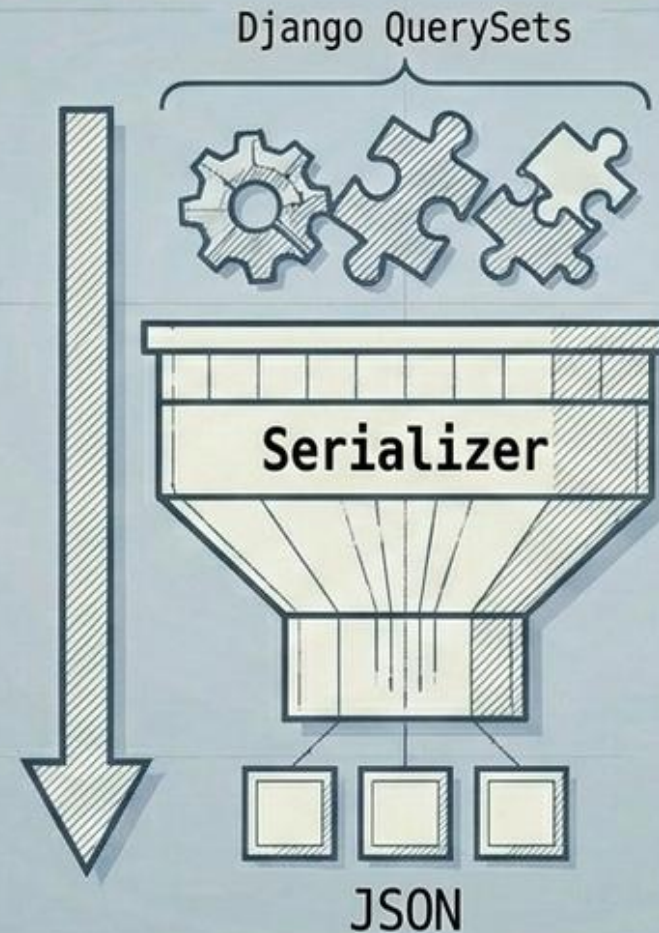
JSON is similar to the syntax for creating JavaScript objects.

JSON is used to **send, receive** and **store data**.

[ESTUDE OU REVISE JSON AQUI](#)

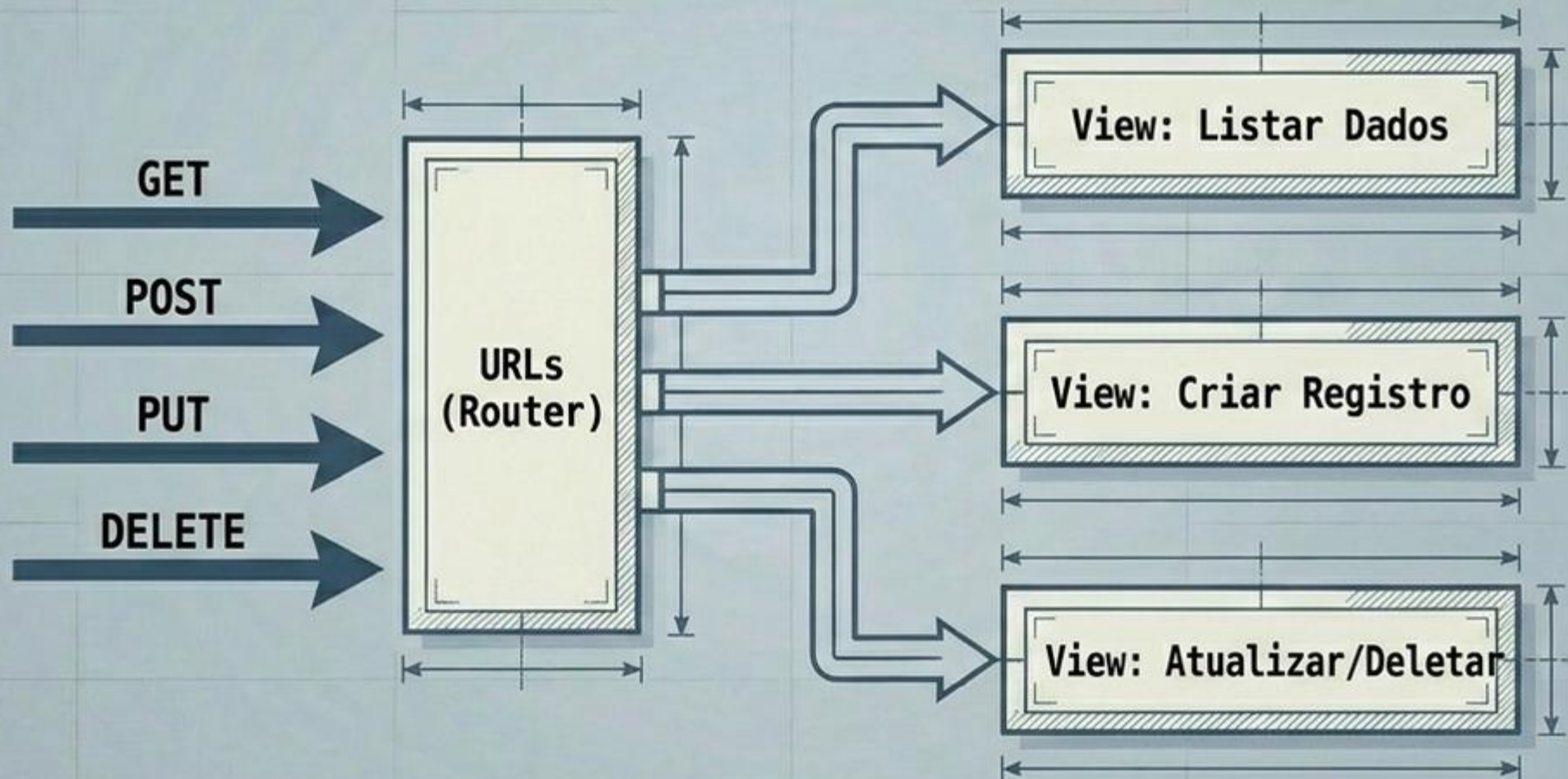
Serializers: O Funil de Conversão

Objetos complexos de banco de dados não trafegam bem pela internet. Os serializadores atuam como tradutores rigorosos: convertem instâncias do Django em tipos de dados nativos do Python que podem ser facilmente renderizados em JSON.



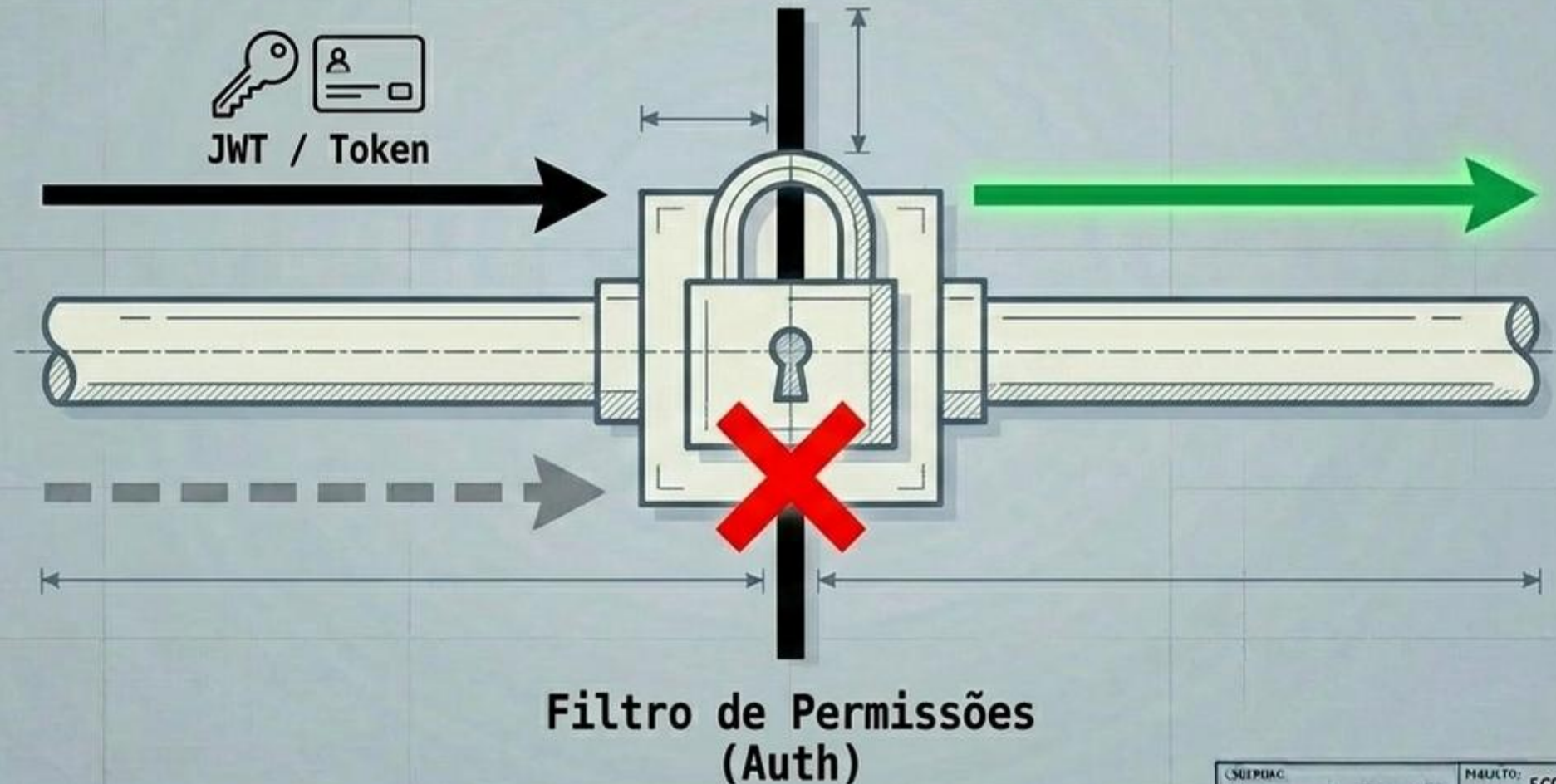
Views e URLs: O Roteamento Inteligente

As URLs determinam onde a requisição deve ir. As Views determinam o que deve acontecer quando ela chega. Desenvolveremos a lógica central que processa os dados, aplica regras de negócio e formula a resposta HTTP correta.



Autenticação: A Camada de Segurança

APIs públicas são vulneráveis. Implementaremos sistemas de autenticação robustos para garantir que nossas rotas estejam protegidas e que apenas usuários verificados tenham acesso aos recursos do sistema.



Síntese Prática: A API Escolar

A teoria apenas ganha vida através da prática. Aplicaremos todos os conceitos de arquitetura, roteamento e serialização na construção de um sistema completo e funcional: uma API de gestão escolar desenvolvida do zero absoluto.



```
(.venv) PS C:\Users\ALUNO\Desktop\apischool> python --version  
Python 3.13.0
```

```
(.venv) PS C:\Users\ALUNO\Desktop\apischool> pip list
```

Package	Version
-----	-----
asgiref	3.11.1
certifi	2026.5.20
charset-normalizer	3.4.7
Django	5.1.3 ←

Django REST Framework (DRF)

- O Django REST Framework (DRF) é uma poderosa extensão do framework web Django para Python, projetada especificamente para facilitar a criação de APIs RESTful de forma eficiente e escalável.
- Assim como Python é conhecido por sua simplicidade e versatilidade, este framework herda tais características, permitindo às pessoas desenvolvedoras construir APIs robustas com facilidade, seguindo as melhores práticas da arquitetura REST.

django

REST

framework

Table of contents

[Requirements](#)

[Installation](#)

[Example](#)

[Quickstart](#)

[Development](#)

[Support](#)

[Security](#)

[License](#)

Ok. Hands On

1. `django-admin startproject setup` .

```
1 | from django.db import models
```

Model - Estudantes

Etiquetas

Model

Descrição

Dados necessários:

- Id
- Nome
- E-mail
 - Não pode estar em branco
- CPF
 - Máximo de 11 caracteres
- Data de Nascimento
- Número de Celular
 - Máximo de 14 caracteres

Model - Cursos

Etiquetas

Model

Descrição

Dados necessários:

- Id
- Código
 - Máximo de 10 caracteres
- Descrição
 - Não pode estar em Branco
- Nível (Básico, Intermediário e Avançado)
 - Não pode estar em Branco
 - Não pode ser Nulo
 - Por padrão deve ser Básico

Model - Matriculas

Etiquetas

Model

Descrição

Dados necessários:

- Id
- Relacionar com o modelo Estudante
 - Se o Estudante for deletado, todas as matrículas daquele estudante devem ser deletadas
- Relacionar com o modelo Curso
 - Se o Curso for deletado, todas as matrículas daquele curso devem ser deletadas
- Período (Matutino, Vespertino, Noturno)
 - Não pode estar em Branco
 - Não pode ser Nulo
 - Por padrão deve ser Matutino

```
4 class Estudante(models.Model):
5     """Model que representa um estudante."""
6
7     nome = models.CharField(max_length=255)
8     email = models.EmailField(blank=False)
9     cpf = models.CharField(max_length=11)
10    data_nascimento = models.DateField()
11    numero_celular = models.CharField(max_length=14)
12
13    class Meta:
14        verbose_name = "Estudante"
15        verbose_name_plural = "Estudantes"
16
17    def __str__(self):
18        return self.nome
19
```

```
21 class Curso(models.Model):
22     """Model que representa um curso."""
23
24     class Nivel(models.TextChoices):
25         BASICO = "Basico", "Básico"
26         INTERMEDIARIO = "Intermediario", "Intermediário"
27         AVANCADO = "Avancado", "Avançado"
28
29     codigo = models.CharField(max_length=10)
30     descricao = models.TextField(blank=False)
31     nivel = models.CharField(
32         max_length=15,
33         choices=Nivel.choices,
34         default=Nivel.BASICO,
35         blank=False,
36         null=False,
37     )
```

Continuação da classe Curso

```
39     class Meta:
40         verbose_name = "Curso"
41         verbose_name_plural = "Cursos"
42
43     def __str__(self):
44         return f"{self.codigo} - {self.descricao}"
45
```



```
47 class Matricula(models.Model):
48     """Model que representa a matrícula de um estudante em um curso."""
49
50     class Periodo(models.TextChoices):
51         MATUTINO = "Matutino", "Matutino"
52         VESPERTINO = "Vespertino", "Vespertino"
53         NOTURNO = "Noturno", "Noturno"
54
55     estudante = models.ForeignKey(
56         Estudante,
57         on_delete=models.CASCADE,
58         related_name="matriculas",
59     )
60     curso = models.ForeignKey(
61         Curso,
62         on_delete=models.CASCADE,
63         related_name="matriculas",
64     )
```

```
65     periodo = models.CharField(
66         max_length=10,
67         choices=Periodo.choices,
68         default=Periodo.MATUTINO,
69         blank=False,
70         null=False,
71     )
72
73     class Meta:
74         verbose_name = "Matrícula"
75         verbose_name_plural = "Matrículas"
76
77     def __str__(self):
78         return f"{self.estudiante} → {self.curso} ({self.periodo})"
79
```